📄 DayScore — Technical Documentation

Document Version: 2.0
Date: 27 March 2026
Author: DayScore Development Team
Framework: Streamlit + Firebase + Google Fit
Python Version: 3.12+

Table of Contents

1. Project Overview
2. Python Libraries & Dependencies
3. Project Structure
4. File-by-File Documentation
5. Complete Code Flow
6. Authentication Flow
7. Data Flow & Firestore Schema
8. DayScore Calculation Algorithm
9. API Integrations
10. Security Architecture
11. Performance Optimizations
12. Environment Setup
13. Deployment Guide

---

1. Project Overview

DayScore is a comprehensive wellness tracking web application that:

- Connects to **Google Fit** to pull health metrics (steps, sleep, calories, heart rate)
- Calculates a **daily wellness score (0–100)** using a weighted algorithm
- Provides **AI-powered insights** via OpenAI GPT-3.5
- Offers **mood journaling**, **mental health tracking**, and **counselor booking**
- Encourages healthy habits through **gamification** (streaks, badges, challenges)
- Supports **Demo Mode** for use without any API keys

Key Statistics

Metric	Value
Total Python Files	41
Total Source Code	273.7 KB
Services	12
Feature Pages	11
Firestore Collections	9
Dependencies	14 (all version-pinned)

---

2. Python Libraries & Dependencies

Core Libraries

#	Library	Version	Purpose	Used In
1	**streamlit**	≥1.40.0	Web UI framework — renders all pages, handles routing via `st.navigation`, manages session state, caching	Every file
2	**python-dotenv**	≥1.0.0	Loads environment variables from `.env` file into `os.environ`	`config/settings.py`

Firebase & Google

#	Library	Version	Purpose	Used In
3	**firebase-admin**	≥6.5.0	Server-side Firebase SDK — Firestore database, user authentication, Storage	`config/firebase_config.py`, all services
4	**google-auth**	≥2.28.0	Google OAuth2 credentials management — token refresh, validation	`auth/google_oauth.py`, `services/google_fit_service.py`
5	**google-auth-oauthlib**	≥1.2.0	OAuth2 flow helpers — authorization URL generation, token exchange	`auth/google_oauth.py`
6	**google-auth-httplib2**	≥0.2.0	HTTP transport layer for Google API authentication	`services/google_fit_service.py`
7	**google-api-python-client**	≥2.120.0	Google Fit REST API client — fetches health data (steps, sleep, calories, heart rate)	`services/google_fit_service.py`

AI & Machine Learning

#	Library	Version	Purpose	Used In
8	**openai**	≥1.12.0	OpenAI GPT-3.5-turbo API — powers the AI wellness chatbot conversations	`services/chatbot_service.py`
9	**scikit-learn**	≥1.4.0	Machine learning — Linear Regression model for next-day DayScore prediction	`services/prediction_service.py`
10	**numpy**	≥1.26.0	Numerical computing — array operations for ML feature vectors	`services/prediction_service.py`, `services/scoring_service.py`

11	**pandas**	≥2.2.0	Data manipulation — DataFrames for historical score analysis and trend computation	`services/prediction_service.py`

Visualization & Utilities

#	Library	Version	Purpose	Used In
12	**plotly**	≥5.18.0	Interactive charts — gauge, bar, line, and area charts on dashboard and analytics	`features/dashboard.py`, `features/results.py`, `features/mental_health.py`
13	**requests**	≥2.31.0	HTTP client — REST API calls to Firebase Auth, Google token endpoints, image URLs	`auth/auth_service.py`, `auth/google_oauth.py`, `utils/helpers.py`
14	**Pillow**	≥10.2.0	Image processing — profile photo resizing and format conversion	`utils/helpers.py`

Standard Library (No Installation Required)

Library	Purpose	Used In
`datetime`, `timedelta`, `timezone`	Timezone-aware timestamps for all data	All services
`concurrent.futures`	`ThreadPoolExecutor` for parallel API calls	`services/google_fit_service.py`
`secrets`	CSRF token generation for OAuth security	`auth/google_oauth.py`
`json`, `os`	Config parsing, environment variables	`config/settings.py`, `auth/google_oauth.py`
`random`	Demo data generation, counselor IDs	Multiple services
`smtplib`, `email.message`	SMTP email for counselor booking confirmations	`services/counselor_service.py`
`hashlib`	Password hashing utilities	`auth/auth_service.py`
`re`	Input validation patterns	`auth/auth_service.py`

---

3. Project Structure

```
dayscore-full/                        ← Root directory
│
├── app.py                             ← 🚀 MAIN ENTRY POINT
├── requirements.txt                   ← Version-pinned dependencies
├── .env.example                       ← Environment variable template
├── .gitignore                         ← Git exclusions
│
```

```
├── .streamlit/
│   └── config.toml                    ← Streamlit theme (colors, fonts)
│
├── config/                            ← ⚙️ CONFIGURATION LAYER
│   ├── __init__.py
│   ├── settings.py                    ← API keys, defaults, constants
│   ├── firebase_config.py             ← Firebase Admin SDK init
│   └── serviceAccountKey.json         ← Firebase credentials (gitignored)
│
├── auth/                              ← 🔐 AUTHENTICATION LAYER
│   ├── __init__.py
│   ├── auth_service.py                ← Email/Password + Google sign-in
│   └── google_oauth.py                ← OAuth 2.0 flow + CSRF protection
│
├── services/                          ← 🧠 BUSINESS LOGIC LAYER
│   ├── __init__.py
│   ├── scoring_service.py             ← DayScore algorithm (0-100)
│   ├── google_fit_service.py          ← Google Fit API integration
│   ├── achievement_service.py         ← Badge & achievement system
│   ├── challenge_service.py           ← Community challenges
│   ├── chatbot_service.py             ← AI chatbot (OpenAI + fallback)
│   ├── counselor_service.py           ← Counselor directory & booking
│   ├── journal_service.py             ← Mood journaling & sentiment
│   ├── messaging_service.py           ← User-to-user messaging
│   ├── notification_service.py        ← Smart reminders & alerts
│   ├── prediction_service.py          ← ML score prediction
│   ├── streak_service.py              ← Streak tracking
│   └── wellness_service.py            ← Wellness exercise data
│
├── features/                          ← 📱 UI / PAGE LAYER
│   ├── __init__.py
│   ├── dashboard.py                   ← Dashboard page
│   ├── results.py                     ← Results & Analytics page
│   ├── mental_health.py               ← Mental Health Tracker page
│   ├── journal.py                     ← Journal page
│   ├── achievements.py                ← Achievements page
│   ├── challenges.py                  ← Community Challenges page
│   ├── wellness.py                    ← Wellness Center page
│   ├── counselor.py                   ← Counselor Directory page
│   ├── messages.py                    ← Messaging page
│   ├── settings.py                    ← Settings page
│   └── chat/                          ← AI Chatbot sub-module
│       ├── main.py                    ← Chatbot entry point
│       ├── chat_logic.py              ← Conversation handling
│       └── chat_ui.py                 ← Chat interface components
│
└── utils/                             ← 🔧 UTILITIES LAYER
    ├── __init__.py
    ├── session_manager.py             ← Type-safe session state wrapper
    ├── ui_components.py               ← Reusable CSS & UI components
    └── helpers.py                     ← Formatters, validators, loaders
```

---

4. File-by-File Documentation

4.1 Root Files

File	Size	Purpose
app.py	13.6 KB	**Main entry point.** Configures `st.set_page_config()`, injects CSS, runs the auth gate (login/signup/demo), and builds `st.navigation()` with all 11 pages. Handles Google OAuth callback, Google Fit credential restoration, and sidebar rendering.
requirements.txt	0.3 KB	All 14 Python package dependencies, version-pinned with `>=` minimum versions.
.env.example	0.4 KB	Template for environment variables: Firebase config, Google OAuth Client ID/Secret, OpenAI API key, SMTP settings.

4.2 Config Layer (`config/`)

File	Size	Purpose
settings.py	3.4 KB	Central configuration hub. Defines `AppConfig` (app name, version), `FirebaseConfig` (API key, project ID), `GoogleFitConfig` (client ID, secret, scopes), `OpenAIConfig` (API key, model), `SMTPConfig` (email server). Loads values from `st.secrets` → `.env` → defaults.
firebase_config.py	3.3 KB	Firebase Admin SDK initialization. Uses `@st.cache_resource` (singleton pattern) to create Firestore client, Auth instance, and Storage bucket. Loads `serviceAccountKey.json`. Returns `None` in demo mode for graceful fallback.

4.3 Auth Layer (`auth/`)

File	Size	Purpose
auth_service.py	10.7 KB	Core authentication service. Handles: (1) Email/Password sign-up via Firebase Auth REST API, (2) Email/Password sign-in with token exchange, (3) Google Sign-In user creation/linking, (4) Password reset email, (5) Demo mode login with simulated user. Stores user profile in Firestore `users` collection with timestamps using `datetime.now(timezone.utc)`.
google_oauth.py	6.1 KB	Google OAuth 2.0 implementation. Generates authorization URL with CSRF token (`secrets.token_urlsafe(32)`) in `state` parameter. Handles callback: validates CSRF token, exchanges auth code for tokens, fetches user profile from Google People API. Separates Sign-In scopes from Fit scopes.

4.4 Services Layer (`services/`) — Business Logic

File	Size	Purpose
scoring_service.py	5.2 KB	DayScore calculation engine. Computes individual scores for steps (30%), sleep (30%), calories (20%), heart rate (20%). Each metric has a sub-

		algorithm (e.g., sleep peaks at 7–8h, tapers outside). Returns 0–100 score + per-metric breakdown + personalized tips.
google_fit_service.py	10.4 KB	Google Fit API integration. Fetches daily health data (steps, sleep, calories, heart rate) from Google Fit REST API. Uses `ThreadPoolExecutor` for parallel metric fetching (4 metrics simultaneously) and parallel weekly fetching (7 days at once). Falls back to demo data if API unavailable. Syncs daily data to Firestore.
prediction_service.py	5.5 KB	ML-based score prediction. Trains a `LinearRegression` model (scikit-learn) on the user's past 30 days of Firestore score history. Engineers features (day-of-week, rolling averages). Predicts tomorrow's DayScore with confidence interval. Falls back to statistical average if insufficient data.
chatbot_service.py	8.4 KB	Multi-personality AI chatbot. Supports 4 personas: Motivator, Therapist, Fitness Coach, Nutritionist. Calls OpenAI's `chat.completions.create()` with system prompts per persona. Falls back to intelligent rule-based responses if no API key. Stores conversation history in Firestore `chat_history` collection.
journal_service.py	9.9 KB	Mood journaling service. Provides a 5-level mood scale (😢 → 😄). Stores journal entries in Firestore with mood, emotion tags, gratitude text, and timestamps. Supports retrieval by date range using `FieldFilter` queries. Runs basic sentiment analysis on entry text.
counselor_service.py	18.2 KB	Counselor directory and booking system. Contains 6 hardcoded counselor profiles with specialties, tags, ratings. Supports filtering (type-safe `str
challenge_service.py	7.4 KB	Community challenges. Create challenges with metric targets (steps, sleep, etc.) and date ranges. Join challenges, submit daily progress, query leaderboard. Checks challenge expiry with timezone-aware timestamps.
achievement_service.py	7.1 KB	Achievement and badge system. Defines 10 achievement types with conditions. Checks eligibility based on user data (score thresholds, streak lengths, step counts). Awards badges and stores in Firestore `achievements` collection.
streak_service.py	4.4 KB	Streak tracking. Calculates current streak (consecutive days with a DayScore entry), longest streak, and weekly consistency. Uses `datetime.now(timezone.utc)` for accurate day boundary detection.
messaging_service.py	4.6 KB	User-to-user messaging. Send messages with Firestore real-time storage. Query conversation threads by sender/receiver. Marks messages as read. Uses timezone-aware timestamps for ordering.
notification_service.py	3.7 KB	Smart notification system. Generates contextual reminders based on current metrics (low steps → "take a walk", poor sleep → "sleep hygiene tips"). Detects achievement unlock events and generates celebration messages.
wellness_service.py	1.8 KB	Wellness exercise data provider. Returns structured data for breathing exercises (4-4-6, 4-7-8, Box breathing patterns), mind games, and relaxation sound configurations.

4.5 Features Layer (`features/`) — UI Pages

File	Size	Purpose
dashboard.py	16.1 KB	Main dashboard page. Renders DayScore gauge (Plotly), metric cards (steps/sleep/calories/HR), 7-day trend chart, achievement notifications, streak display, and motivational messages. Fetches data from `GoogleFitService` + `ScoringService`.
results.py	9.5 KB	Results & Analytics page. Shows best/average/worst scores, weekly trend analysis with Plotly bar charts, per-metric breakdowns, and AI-generated improvement tips.
mental_health.py	16.3 KB	Mental Health Tracker. Mood logging with 5-level scale, emotion tagging, Plotly mood trend chart, weekly summary statistics, gratitude prompts, and mental health insights.
journal.py	8.4 KB	Journal page. Daily reflective journaling with date picker, mood selection, emotion tags, gratitude field, and free-form text. Displays past entries with search.
achievements.py	5.1 KB	Achievements page. Displays all 10 badges with lock/unlock status, progress indicators, category grouping, and unlock timestamps.
challenges.py	6.8 KB	Community Challenges page. Create new challenges with metric targets, browse available challenges, join challenges, and view leaderboard with rankings.
wellness.py	18.7 KB	Wellness Center. Three tabs: (1) Breathing exercises with animated circle guide, (2) Mind games (reaction time, memory, number zen), (3) Relaxation sounds (rain, forest, white noise, healing frequencies).
counselor.py	16.4 KB	Counselor Directory. Browse counselors with specialty filter, view detailed profiles, book sessions with date/time picker, rate past sessions, and view booking history.
messages.py	7.7 KB	Messaging page. Conversation list, message thread view, compose new messages, and real-time message display with timestamps.
settings.py	7.1 KB	Settings page. Profile editing (name, bio), Google Fit connection toggle, theme preferences, notification settings, and account management (logout, delete).
chat/ (sub-module)	17.3 KB total	AI Chatbot module split into 3 files: `main.py` (entry point + page routing), `chat_logic.py` (conversation management, persona switching, history), `chat_ui.py` (chat bubble rendering, input handling, personality selector).

4.6 Utils Layer (`utils/`)

File	Size	Purpose
session_manager.py	3.7 KB	Type-safe wrapper around `st.session_state`. Provides Python property accessors (`.is_authenticated`, `.user`, `.google_fit_connected`, etc.) instead of raw

		dict access. Initializes all session keys with safe defaults.
ui_components.py	12.1 KB	Shared UI components and CSS injection. Contains reusable card components, metric displays, and a large custom CSS stylesheet (glassmorphism theme, gradient colors, animations, responsive layout). `inject_custom_css()` is called once from `app.py`.
helpers.py	6.9 KB	Utility functions: date formatting, score color mapping, percentage calculation, image loading from URL with Pillow, emoji helpers, weekly trend label generation, and error display formatting.

---

5. Complete Code Flow

5.1 Application Startup Flow

```
User opens http://localhost:8501
         │
         ▼
    ┌──────────────────────────────┐
    │         app.py (Entry)       │
    │                              │
    │  1. SessionManager.initialize() │
    │  2. st.set_page_config()     │
    │  3. inject_custom_css()      │
    │  4. Check: is_authenticated? │
    └──────────────┬───────────────┘
                   │
        ┌──────────┴──────────┐
        │ NO                  │ YES
        ▼                     ▼
┌──────────────┐    ┌──────────────────────┐
│  Login Page  │    │   Build Navigation   │
│              │    │                      │
│ • Email/Pass │    │  st.navigation([     │
│ • Google SSO │    │    Dashboard,        │
│ • Demo Mode  │    │    Results,          │
│ • Sign Up    │    │    Mental Health,    │
└──────┬───────┘    │    Journal, ...      │
       │            │  ])                  │
       │ Auth OK    │                      │
       ▼            │  nav.run()           │
  Set session:      └──────────────────────┘
  • is_authenticated = True
  • user = {...}
  • token = "..."
  st.rerun()
```

5.2 Page Rendering Flow

```
st.navigation selects page (e.g. Dashboard)
         │
         ▼
┌──────────────────────────────────────┐
│        features/dashboard.py         │
│                                      │
│  1. Get user_id from SessionManager  │
│  2. Initialize services:             │
│     • GoogleFitService()             │
│     • ScoringService()               │
│     • StreakService()                 |
│     • AchievementService()           │
│                                      │
│  3. Fetch today's fitness data:      │
│     fit_service.fetch_daily_data()   │
│       → ThreadPoolExecutor (4 threads) |
│       → Steps, Sleep, Calories, HR   │
│       → Returns dict                 │
│                                      │
│  4. Calculate DayScore:              │
│     scoring_service.calculate_score() |
│       → Weighted algorithm (0-100)   │
│       → Returns score + breakdown    │
│                                      │
│  5. Render UI:                       │
│     • Plotly gauge chart             │
│     • Metric cards (4 columns)       │
│     • Weekly trend bar chart         │
│     • Streak counter                 │
│     • Achievement notifications      │
│     • Motivational message           │
│                                      │
│  6. Save to Firestore:               │
│     daily_scores/{date}              │
└──────────────────────────────────────┘
```

5.3 Data Fetching Flow (Google Fit)

```
GoogleFitService.fetch_daily_data(credentials, date)
         │
         ▼
┌──────────────────────────────────────┐
│  1. Build OAuth2 Credentials object  │
│  2. Create Fitness API client        │
│  3. Calculate time range (start/end) │
│                                      │
│  4. ThreadPoolExecutor(max_workers=4) |
│     ├─ Thread 1: fetch_steps()       │
│     ├─ Thread 2: fetch_calories()    │
│     ├─ Thread 3: fetch_sleep()       │
│     └─ Thread 4: fetch_heart_rate()  |
```

```
│                                        │
│  5. Collect results from all threads   │
│  6. Return combined dict:              │
│     {                                  │
│       "steps": 8432,                   │
│       "sleep_hours": 7.2,             │
│       "calories": 1856,               │
│       "heart_rate": 72,                │
│       "date": "2026-03-27"            │
│     }                                  │
└────────────────────────────────────────┘
```

5.4 AI Chatbot Flow

```
User sends message
         │
         ▼
┌────────────────────────────────────┐
│  chat/chat_logic.py                 │
│                                     │
│  1. Load conversation history       │
│     from Firestore chat_history     │
│                                     │
│  2. Build messages array:           │
│     [                               │
│       {system: persona_prompt},     │
│       {user: msg_1},               │
│       {assistant: reply_1},        │
│       ...                           │
│       {user: current_message}       │
│     ]                               │
│                                     │
│  3. Call OpenAI API:                │
│     openai.chat.completions.create() │
│     model: "gpt-3.5-turbo"         │
│                                     │
│  4. If OpenAI fails → rule-based    │
│     fallback with keyword matching  │
│                                     │
│  5. Save to Firestore:             │
│     chat_history/{user_id}/messages │
│                                     │
│  6. Display in chat_ui.py           │
│     with styled chat bubbles        │
└────────────────────────────────────┘
```

---

6. Authentication Flow

Email/Password Authentication

```
User enters email + password
         │
         ▼
auth_service.sign_in(email, password)
         │
         ▼
POST → Firebase Auth REST API
       identitytoolkit.googleapis.com
         │
         ▼
Returns: { idToken, localId, email }
         │
         ▼
Fetch user profile from Firestore users/{localId}
         │
         ▼
Store in SessionManager:
  • is_authenticated = True
  • user = { uid, email, name, ... }
  • token = idToken
```

Google OAuth Authentication

```
User clicks "Sign in with Google"
         │
         ▼
google_oauth.get_google_auth_url()
  → Generate CSRF token (secrets.token_urlsafe(32))
  → Store in st.session_state["_oauth_csrf_token"]
  → Build auth URL with state="google_signin_{token}"
         │
         ▼
Redirect to Google Consent Screen
         │
         ▼
Google redirects back with ?code=xxx&state=google_signin_{token}
         │
         ▼
google_oauth.handle_google_callback()
  → Verify CSRF: received token == stored token
  → Exchange code for access_token + refresh_token
  → Fetch user profile from Google People API
  → Return { email, name, picture, tokens }
         │
         ▼
auth_service.sign_in_with_google(user_info)
  → Create/update Firestore user document
  → Store Google Fit credentials for later use
```

---

7. Data Flow & Firestore Schema

Firestore Collections

Collection	Document ID	Key Fields
`users`	`{user_id}`	`email`, `name`, `created_at`, `google_fit_credentials`, `streak`, `points`, `level`
`daily_scores`	Auto-generated	`user_id`, `date`, `total_score`, `steps`, `sleep_hours`, `calories`, `heart_rate`, `timestamp`
`chat_history`	Auto-generated	`user_id`, `role` (user/assistant), `content`, `persona`, `timestamp`
`journal_entries`	Auto-generated	`user_id`, `date`, `mood_value`, `mood_label`, `emotions`, `gratitude`, `text`, `timestamp`
`challenges`	Auto-generated	`title`, `creator_id`, `metric`, `target`, `start_date`, `end_date`, `participants`
`leaderboard`	Auto-generated	`challenge_id`, `user_id`, `score`, `rank`
`achievements`	Auto-generated	`user_id`, `badge_id`, `badge_name`, `unlocked_at`
`messages`	Auto-generated	`sender_id`, `receiver_id`, `content`, `timestamp`, `read`
`counselor_bookings`	Auto-generated	`user_id`, `counselor_id`, `date`, `time`, `status`, `rating`, `review`

---

8. DayScore Calculation Algorithm

```python
# services/scoring_service.py

def calculate_score(steps, sleep_hours, calories, heart_rate):

    # Step 1: Calculate individual metric scores (0-100)
    step_score     = min(steps / 10000 * 100, 100)        # Linear, cap at 10K
    sleep_score    = bell_curve(sleep_hours, peak=7.5)     # Peak at 7-8h
    calorie_score  = min(calories / 2000 * 100, 100)      # Linear, cap at 2K
    hr_score       = hr_zone_score(heart_rate)             # Best at 60-70 BPM

    # Step 2: Apply weights
    weights = {
        'steps':      0.30,   # 30%
        'sleep':      0.30,   # 30%
        'calories':   0.20,   # 20%
        'heart_rate': 0.20,   # 20%
    }
```

```
    # Step 3: Weighted sum
    total = (step_score * 0.30 +
             sleep_score * 0.30 +
             calorie_score * 0.20 +
             hr_score * 0.20)

    # Step 4: Clamp to 0-100
    return max(0, min(100, round(total)))
```

---

9. API Integrations

Google Fit API

Endpoint	Data	Method
`fitness.googleapis.com/v1/users/me/dataSources`	Steps	`dataStreamId` aggregate
`fitness.googleapis.com/v1/users/me/dataSources`	Calories	`dataStreamId` aggregate
`fitness.googleapis.com/v1/users/me/sessions`	Sleep	Session-based query
`fitness.googleapis.com/v1/users/me/dataSources`	Heart Rate	`dataStreamId` aggregate

OpenAI API

Parameter	Value
Model	`gpt-3.5-turbo`
Endpoint	`chat.completions.create()`
System Prompts	4 personas (Motivator, Therapist, Coach, Nutritionist)
Fallback	Rule-based keyword matching

Firebase Auth REST API

Endpoint	Purpose
`identitytoolkit.googleapis.com/v1/accounts:signInWithPassword`	Email/password login
`identitytoolkit.googleapis.com/v1/accounts:signUp`	New user registration
`identitytoolkit.googleapis.com/v1/accounts:sendOobCode`	Password reset email

---

10. Security Architecture

Security Feature	Implementation
Authentication	Firebase Admin SDK (server-side token verification)
OAuth CSRF	`secrets.token_urlsafe(32)` in OAuth `state` parameter

Credential Storage	`.env` file + `st.secrets` (never committed to git)
Service Account	`serviceAccountKey.json` in `.gitignore`
Datetime Safety	All timestamps use `datetime.now(timezone.utc)` (Python 3.12+ compliant)
Type Safety	`str
Dependency Pinning	All 14 packages version-pinned with `>=`

---

11. Performance Optimizations

Optimization	Implementation	Impact
Parallel Metric Fetch	`ThreadPoolExecutor(max_workers=4)` for 4 fitness metrics	~4× faster daily data
Parallel Weekly Fetch	`ThreadPoolExecutor(max_workers=4)` for 7 days	~7× faster weekly data
API Response Caching	`@st.cache_data(ttl=300)` on Fit API calls	5-min cache
Singleton Clients	`@st.cache_resource` for Firebase/Auth/Storage	One init per session
Native Navigation	`st.navigation()` instead of page reload	No full page refreshes
FieldFilter Queries	`FieldFilter()` instead of positional `.where()` args	No deprecation overhead

---

12. Environment Setup

Required Environment Variables (`.env`)

```
# Firebase
FIREBASE_API_KEY=AIzaSy...
FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_STORAGE_BUCKET=your-project.appspot.com
FIREBASE_MESSAGING_SENDER_ID=123456789
FIREBASE_APP_ID=1:123456789:web:abc123

# Google Fit OAuth
GOOGLE_CLIENT_ID=123456789.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxx
GOOGLE_REDIRECT_URI=http://localhost:8501

# OpenAI (Optional)
OPENAI_API_KEY=sk-...

# SMTP (Optional — for counselor booking emails)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_EMAIL=your-email@gmail.com
SMTP_PASSWORD=app-password
```

13. Deployment Guide

Streamlit Community Cloud

1. Push code to GitHub (ensure `.gitignore` excludes secrets)
2. Go to share.streamlit.io
3. Connect repo → Set `app.py` as entry point
4. Add all `.env` values to **Secrets** in TOML format
5. Deploy 🚀

Required Firestore Index

Create a composite index for the prediction service query:

```
Collection: daily_scores
Fields:     user_id (ASC), timestamp (ASC)
```

> *Firebase will auto-prompt with a link when the query first runs.*